

Data Structure

Level 1

Doubly Linked List Implementation

The background of the slide features a complex, abstract network of thin, dark lines crisscrossing over a lighter, textured surface. In the lower portion of the image, there are dark silhouettes of several people, some standing and some appearing to be in a discussion or collaborative work, adding a human element to the technical theme.

Data Structure

Level 1

Doubly Linked List Implementation



Definition

Doubly Linked List (DLL):

A linear data structure where each node contains:

■ Three parts:

■ value → stores the data

■ next → pointer to the next node

■ prev → pointer to the previous node

This allows traversal forward and backward.



Explanation

- ◆ The program performs the following actions:
 - Creates a Node class
 - Allocates 3 nodes dynamically using new
 - Assigns values to each node
 - Connects nodes using next and prev pointers
 - Traverses the list forward and prints values

Procedures

- 1 Define a Node class with value, next, and prev.
- 2 Create pointers: Node1, Node2, Node3.
- 3 Allocate memory using new.
- 4 Assign values (1, 2, 3).
- 5 Link nodes forward using next.
- 6 Link nodes backward using prev.
- 7 Set head = Node1.
- 8 Traverse using while (head != NULL).
- 9 Print values using cout.

Example

Structure in memory:

$\text{NULL} \leftarrow 1 \rightleftharpoons 2 \rightleftharpoons 3 \rightarrow \text{NULL}$

 Result (Output):

1
2
3



Note

The first node's prev must be NULL.
The last node's next must be NULL.



Comparison



Singly Linked List

- ✓ Each node has 1 pointer (next)
- ✓ Can traverse only forward
- ✓ Uses less memory
- ✓ Deleting or inserting at a specific position requires traversal from head



Doubly Linked List

- ✓ Each node has 2 pointers (next and prev)
- ✓ Can traverse forward and backward
- ✓ Uses more memory
- ✓ Deleting or inserting at a specific position is easier because you can access the previous node directly

Warning

- ◆ Memory allocated with `new` is not deleted in this program.
This may cause a memory leak.
Always use `delete` when dynamic memory is no longer needed.

Summary

- ✓ Each node contains data + two pointers
- ✓ Supports forward and backward traversal
- ✓ Uses dynamic memory allocation
- ✓ More flexible than singly linked lists
- ⊘ Requires careful memory management

Thank you 😊